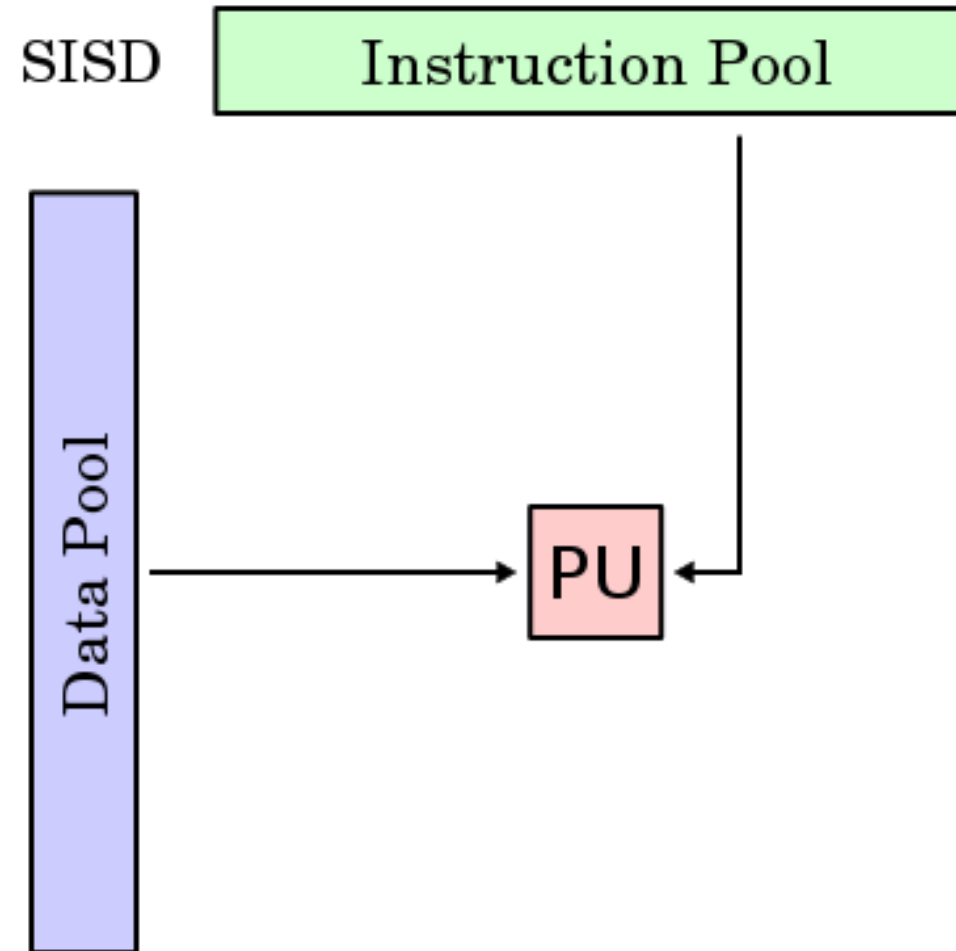


ARQUITETURAS PARALELAS

- Objetivos desse tópico:
 - Identificar as diversas classes de arquiteturas paralelas de hardware definidas pela Taxonomia de Flynn [Tarefa 13]
 - Compreender o impacto e aprender a configurar as diversas formas de afinidade de threads [Tarefa 13]

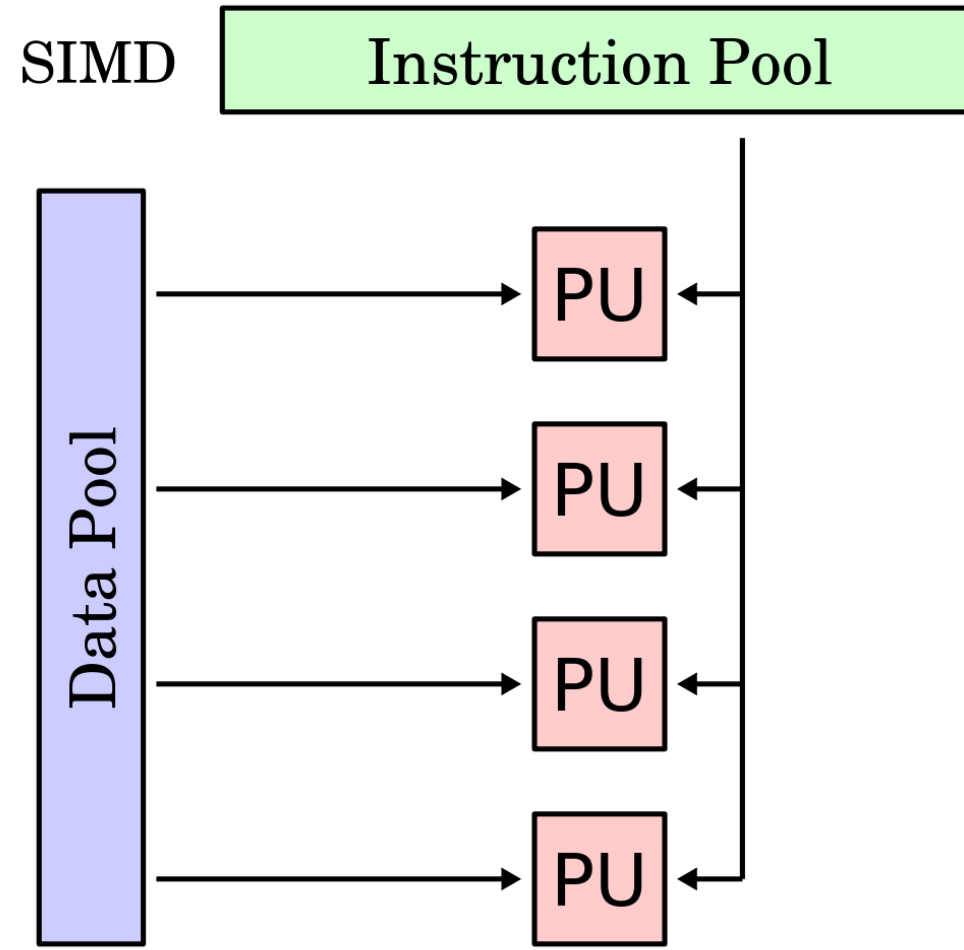
ARQUITETURAS PARALELAS

- Single Instruction Single Data



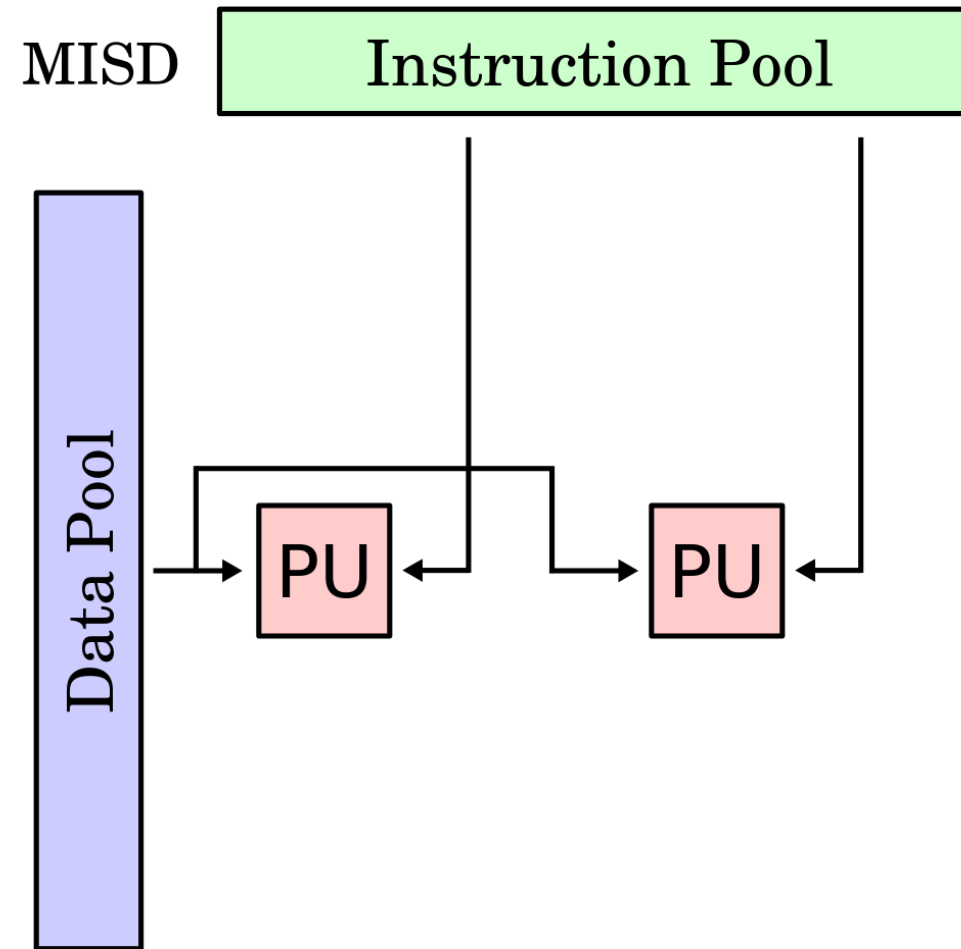
ARQUITETURAS PARALELAS

- Single Instruction Multiple Data



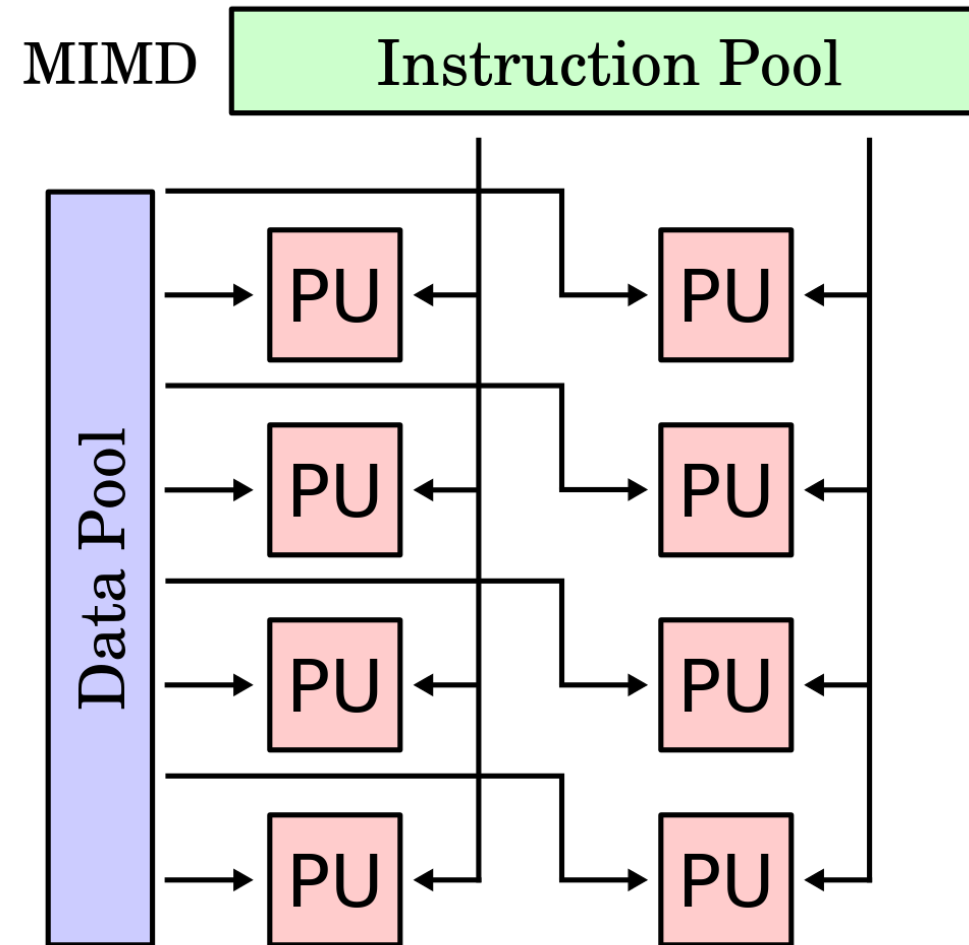
ARQUITETURAS PARALELAS

- Multiple Instruction Single Data



ARQUITETURAS PARALELAS

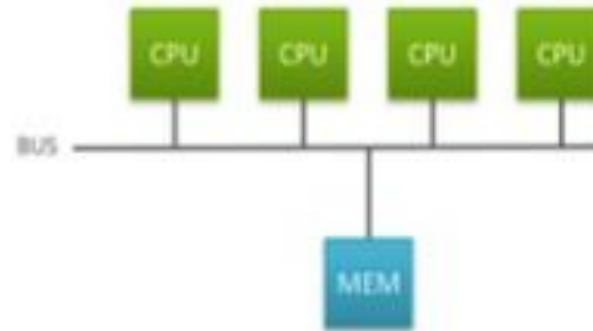
- Multiple Instruction Multiple Data



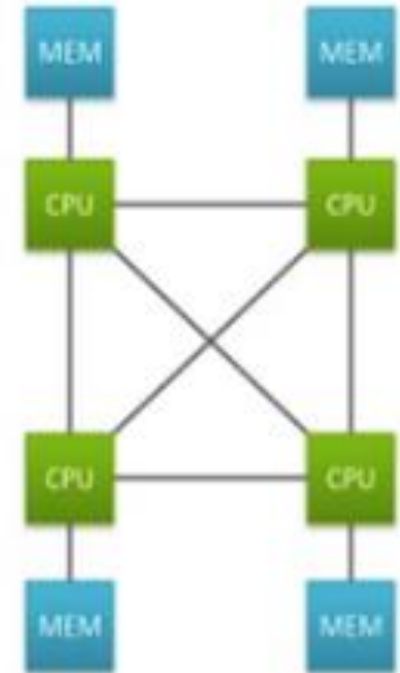
ARQUITETURAS PARALELAS

- Arquiteturas típicas MIMD de memória compartilhada:

Uniform Memory Access (UMA)

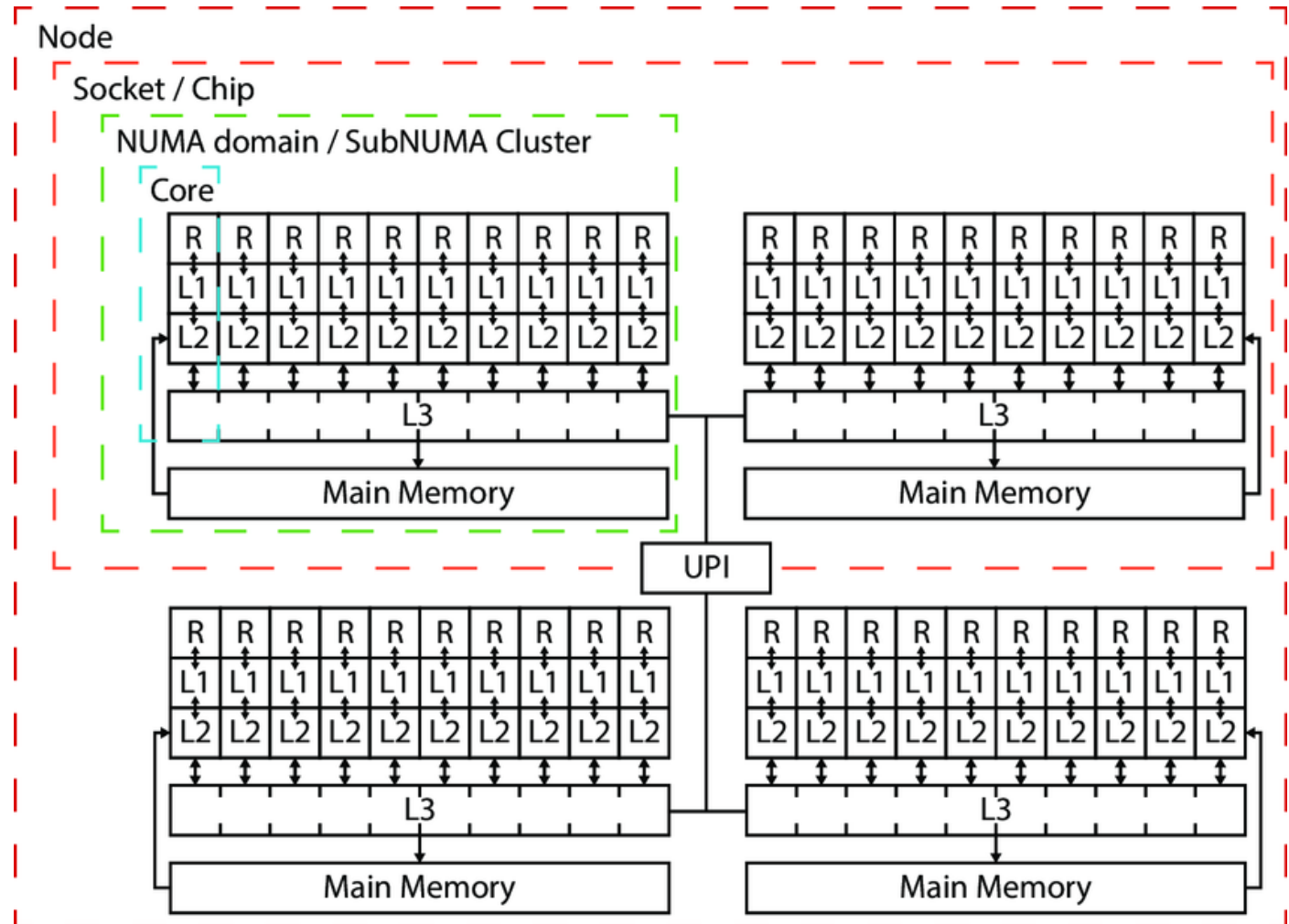


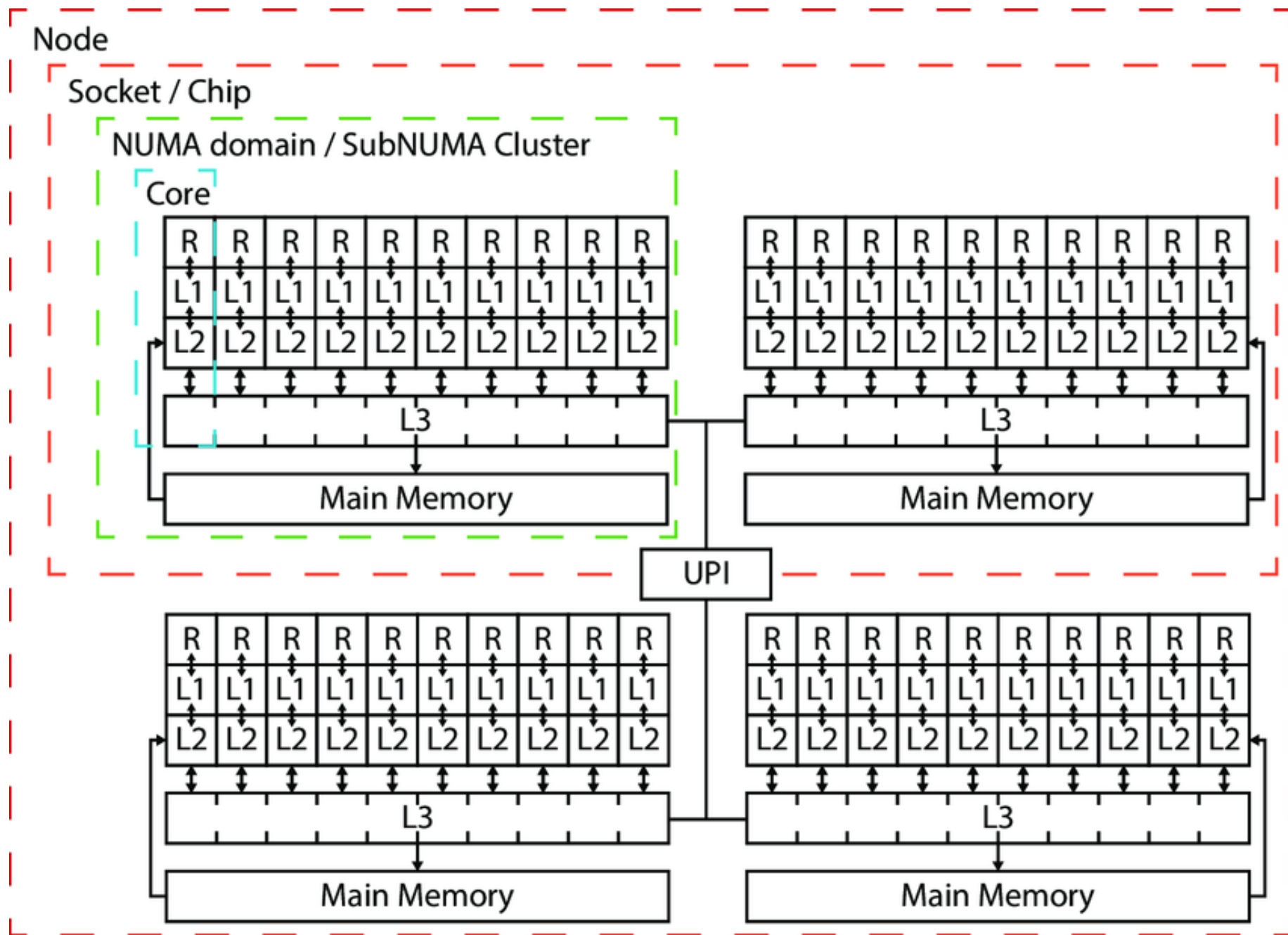
Non-Uniform Memory Access (NUMA)



ARQUITETURAS PARALELAS

- Hierarquia de cache em sistemas NUMA:





ARQUITETURAS PARALELAS

Tarefa 13:

Avalie como a escalabilidade do seu código de Navier-Stokes muda ao utilizar os diversos tipos de afinidades de threads suportados pelo sistema operacional e pelo OpenMP no mesmo nó de computação do NPAD que utilizou para a tarefa 12.

- Afinidade de threads com OpenMP
 - Exemplos com definição de políticas:
 - `export OMP_PROC_BIND=TRUE;` ou
 - `#pragma omp parallel proc_bind(spread)`
 - Valores comuns:
 - `false`: sem afinidade (pode migrar entre núcleos).
 - `true`: thread fica presa ao núcleo onde foi criada.
 - `close`: threads próximas da master thread, mas distribuídas.
 - `spread`: threads espalhadas pelos núcleos (boa para usar múltiplos sockets).
 - `master`: todas seguem a thread 0 (pode forçar várias threads no mesmo núcleo).
 - Para mais controle, use `OMP_PLACES`, por exemplo:
 - `OMP_PLACES=cores`
 - `OMP_PLACES="{0:8:1},{8:8:1}"`